

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)

Architecture logicielle & systèmes distribués

[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

Jour 3 — Fondamentaux des systèmes distribués

Objectifs pédagogiques

Comprendre ce qui change quand le code traverse le réseau : latence, pannes, cohérence, protocoles, contrats et résilience.

Temps estimé

≈ 6–7h incluant démos et quiz

Prérequis

Jours 1 et 2 acquis

Table des matières

- 1. Définition
- 2. Réseau & pannes
- 3. Théorème CAP / PACELC
- 4. Idempotence / retries / timeouts
- 5. Communication sync vs async
- 6. Sérialisation et versioning
- 7. API Gateway
- 8. Service Discovery
- 9. Rate Limiting
- 10. Distributed Tracing
- 11. Event Sourcing
- 12. Contract Testing
- 13. Quiz

Qu'est-ce qu'un système distribué ?

Junior → Expert

Un système distribué est un ensemble de composants exécutés sur plusieurs processus ou machines qui **collaborent pour rendre un service unique** à l'utilisateur. Les processus communiquent uniquement par échange de messages à travers un réseau.

Mode enfant 🧸

Au lieu d'avoir une seule grosse boutique qui vend tout, tu ouvres plusieurs petits magasins spécialisés (un pour les livres, un pour les DVD, un pour la livraison) qui doivent se parler correctement. Si l'un ferme, les autres continuent, mais ils doivent gérer l'absence de leur collègue.

Mode pro 🎓

Le distribué introduit : **latence non-nulle, pannes partielles, duplication de messages, cohérence asynchrone**, sécurité réseau et complexité opérationnelle. En échange, il apporte autonomie de déploiement, scalabilité ciblée et alignement par domaine métier.

Les 8 fausses hypothèses du distribué (Fallacies of Distributed Computing)

Peter Deutsch (Sun Microsystems, 1994) a listé 8 croyances erronées que font les développeurs :

1. Le réseau est fiable.
2. La latence est nulle.
3. La bande passante est infinie.
4. Le réseau est sécurisé.
5. La topologie ne change pas.
6. Il n'y a qu'un seul administrateur réseau.
7. Le transport est sans coût.
8. Le réseau est homogène.

Chaque hypothèse fausse est une source potentielle de bugs de production.

⚙️ Expert — Latence p99 et percentiles

La moyenne est un mensonge en systèmes distribués. Mesurez toujours les **percentiles** (p50, p95, p99, p999). Le p99 représente l'expérience des 1% d'utilisateurs les plus mal servis — souvent vos plus gros clients.

```
// Avec Micrometer : enregistrer la latence d'un appel externe
@Service
public class CatalogClientMetered {

    private final MeterRegistry registry;
    private final CatalogClient client;

    public AvailabilityResponse getAvailability(UUID bookId) {
        return registry.timer("catalog.availability",
```

```

        "service", "catalog")
        .recordCallable(() -> client.getAvailability(bookId));
    }
}

```

Dans Grafana, tracez `histogram_quantile(0.99, rate(catalog_availability_seconds_bucket[5m]))` pour voir le p99 en temps réel.

Latence réseau et pannes partielles

Junior

Mid

Dans un monolithe, un appel de méthode échoue vite et localement. Dans un système distribué, un appel réseau peut être **lent, se perdre, être dupliqué ou échouer après traitement partiel**.

⚠ Attention

Le réseau n'est ni instantané ni fiable. Un timeout n'est pas une erreur — c'est de l'**ambiguïté** : on ne sait pas si la requête a été traitée ou non côté serveur.

Exemple OpenFeign côté `loan-service`

```

...

@FeignClient(name = "catalog-service")
public interface CatalogClient {

    @GetMapping("/internal/books/{bookId}/availability")
    AvailabilityResponse getAvailability(@PathVariable UUID bookId);
}

```

Configurer timeout + retry avec Resilience4j

```

...

# application.yml
resilience4j:
  retry:
    instances:
      catalogClient:
        maxAttempts: 3
        waitDuration: 500ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2 # 500ms → 1s → 2s
        retryExceptions:

```

```

- feign.RetryableException
- java.net.ConnectException
ignoreExceptions:
- com.example.BookNotFoundException # 404 = ne pas retenter
timelimiter:
instances:
catalogClient:
timeoutDuration: 2s
cancelRunningFuture: true

```

⚙️ Expert — Jitter pour éviter le Thundering Herd

Sans jitter, 100 services retentent exactement au même instant et enfoncez encore plus un service déjà surchargé. Ajoutez du bruit aléatoire :

```

resilience4j:
  retry:
    instances:
      catalogClient:
        maxAttempts: 4
        waitDuration: 200ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        # Jitter : ajoute ±25% de délai aléatoire
        randomizedWaitFactor: 0.25
        # Résultat : 200ms±50ms → 400ms±100ms → 800ms±200ms

```

C'est l'algorithme dit "Full Jitter" recommandé par AWS dans leur article de référence sur les retries.

● Erreur fréquente

Appeler un service distant sans timeout ni gestion d'erreur. Un service qui ne répond jamais bloque indéfiniment le thread appelant et épuise le pool → votre service devient indisponible en cascade (**Cascading Failure**).

Théorème CAP — et au-delà avec PACELC

Mid

Senior

Le théorème CAP (Brewer, 2000) rappelle qu'en présence d'une **partition réseau** (P), on ne peut pas garantir simultanément **cohérence forte** (C) et **disponibilité totale** (A).

Choix	Conséquence	Exemple
CP — Favoriser cohérence	On refuse certaines requêtes si le doute existe	Solde bancaire, réservation de siège unique
AP — Favoriser disponibilité	On répond parfois avec une donnée pas encore fraîche	Catalogue de produits, fil d'actualité social

Au-delà de CAP : le théorème PACELC

Daniel Abadi (2012) étend CAP : *même sans partition*, on fait un arbitrage entre **latence** (L) et **cohérence** (C).

Formule : `IF Partition THEN {C ou A} ELSE {L ou C}`

Système	En cas de partition	Cas normal	Cas d'usage
DynamoDB	Disponible (A)	Basse latence (L)	Panier e-commerce, sessions
HBase / ZooKeeper	Cohérent (C)	Cohérent (C)	Coordination distribuée
Cassandra	Disponible (A)	Basse latence (L)	Time-series, IoT
PostgreSQL synchrone	Cohérent (C)	Cohérent (C)	Finance, inventaire

🔧 Expert — Niveaux de cohérence Cassandra

Cassandra expose le niveau de cohérence à l'application. On choisit le trade-off par requête :

```
// Cassandra Driver Java – choisir Le Consistency Level
import com.datastax.oss.driver.api.core.ConsistencyLevel;

SimpleStatement stmt = SimpleStatement
    .builder("SELECT balance FROM accounts WHERE id = ?")
    .setConsistencyLevel(ConsistencyLevel.QUORUM) // Majorité de réplikas
    // QUORUM = (N/2)+1 nœuds doivent répondre
    // ONE    = 1 nœud suffit (plus rapide, moins cohérent)
    // ALL    = tous les nœuds (max cohérence, min dispo)
    .build();

Row row = session.execute(stmt).one();
```

Formule du quorum : si N=3 réplikas, QUORUM = 2. Écriture QUORUM + Lecture QUORUM garantit la lecture-after-write même en présence d'une panne.

Idempotence, retries et timeouts

Junior

Mid

Senior

Définition

Une opération **idempotente** peut être exécutée plusieurs fois sans effet supplémentaire indésirable.

Formellement : $f(f(x)) = f(x)$.

-  **GET, PUT, DELETE** — idempotents par convention REST.
-  **POST** — ne l'est pas nativement → utiliser l'**Idempotency-Key**.

Côté client — JavaScript

```
const idempotencyKey = crypto.randomUUID(); // UUID v4 unique par intention

const response = await fetch('/loans', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Idempotency-Key': idempotencyKey // transmis tel quel à chaque retry
  },
  body: JSON.stringify(loanRequestData)
});
```

Côté serveur — Spring Boot avec Redis

```
@PostMapping("/loans")
public ResponseEntity<LoanResponse> create(
    @RequestHeader("Idempotency-Key") String idempotencyKey,
    @Valid @RequestBody CreateLoanRequest request) {

    // 1. Vérifier si la clé a déjà été traitée (Redis)
    Optional<LoanResponse> cached =
        idempotencyStore.findByKey(idempotencyKey);
    if (cached.isPresent()) {
        return ResponseEntity.ok(cached.get()); // même réponse idempotente
    }

    // 2. Verrouillage distribué pour éviter la double exécution concurrent
    // (race condition si deux retries arrivent en même temps)
    String lockKey = "idempotency-lock:" + idempotencyKey;
    boolean acquired = redisLock.tryLock(lockKey, 5, TimeUnit.SECONDS);
    if (!acquired) {
```

```
return ResponseEntity.status(HttpStatus.CONFLICT)
    .header("Retry-After", "1")
    .build();
}

try {
    // Re-check après Lock (double-checked locking pattern)
    cached = idempotencyStore.findByKey(idempotencyKey);
    if (cached.isPresent()) return ResponseEntity.ok(cached.get());

    // 3. Traiter la requête
    LoanResponse loan = loanApplicationService.createLoan(request);

    // 4. Persister la réponse pour les éventuels retries
    idempotencyStore.save(idempotencyKey, loan, Duration.ofHours(24));

    return ResponseEntity.status(HttpStatus.CREATED).body(loan);
} finally {
    redisLock.unlock(lockKey);
}
}
```

✓ Stratégie de retry recommandée (AWS Best Practices)

- **Exponential backoff** : attendre 100ms, 200ms, 400ms... pour ne pas aggraver une surcharge.
- **Jitter** : variation aléatoire $\pm 25\%$ pour éviter le *thundering herd*.
- **Max retries limité** : 3 à 5 maximum.
- Ne retenter que sur des erreurs **transitoires** (timeout, 503) — **jamais** sur un 400 ou 422 (erreur du client).
- Toujours associer retry + idempotency-key pour les opérations non idempotentes.

Communication synchrone vs asynchrone

Junior

Mid

Senior

Expert

☑ Synchrone

L'appelant envoie et **attend activement**. Le thread est bloqué jusqu'au retour ou timeout.

Exemples : REST/HTTP, gRPC.

✓ Simple, résultat immédiat.

⚠ Couplage temporel — si le service cible est down, l'appelant échoue aussi.

Asynchrone

L'appelant publie dans un broker et **continue immédiatement**. Le consommateur traite quand il est prêt.

Exemples : Kafka, RabbitMQ, AWS SQS.

- ✓ Découplage temporel, résilience, absorption des pics.
- ⚠ Cohérence éventuelle, débogage plus complexe.

REST (Representational State Transfer)

Modèle de maturité REST (Richardson Maturity Model)

Niveau	Description	Exemple
0 — POX	HTTP est utilisé comme un simple tunnel de transport pour des appels de procédure distante (RPC) sans se soucier des ressources.	<code>POST /service?action=getBook</code>
1 — Ressources	Les API exposent des ressources distinctes , chacune identifiée par une URL unique. L'accent est mis sur l'identification des "choses" (les noms) plutôt que des "actions".	<code>GET /books/123</code> (récupère la ressource "livre" avec l'ID 123)
2 — Verbes HTTP	Utilisation des méthodes HTTP standard (GET, POST, PUT, DELETE, PATCH) de manière sémantique . Chaque verbe a une intention claire et universellement comprise pour l'action à effectuer sur la ressource.	<code>DELETE /books/123</code> (supprime le livre 123) <code>POST /books</code> (crée un nouveau livre) <code>PUT /books/123</code> (met à jour le livre 123)
3 — HATEOAS	Liens hypermédias dans la réponse	Voir ci-dessous

Niveau 3 — HATEOAS avec Spring HATEOAS

```
// Réponse enrichie de Liens hypermédias
@GetMapping("/loans/{id}")
public EntityModel<LoanDto> getLoan(@PathVariable UUID id) {
    LoanDto loan = loanService.findById(id);

    return EntityModel.of(loan,
        linkTo(methodOn(LoanController.class).getLoan(id)).withSelfRel(),
        linkTo(methodOn(LoanController.class).returnBook(id)).withRel("return"),
        linkTo(methodOn(LoanController.class).renewLoan(id)).withRel("renew"))
}
```



```
);
}
```

```
// Réponse JSON – Le client sait quelles actions sont disponibles
{
  "id": "d290f1ee-6c54-4b01-90e6-d701748f0851",
  "status": "ACTIVE",
  "dueDate": "2024-02-15",
  "_links": {
    "self": { "href": "/loans/d290f1ee-..." },
    "return": { "href": "/loans/d290f1ee-.../return", "method": "POST" },
    "renew": { "href": "/loans/d290f1ee-.../renew", "method": "POST" }
  }
}
```

Le code HATEOAS ci-dessus utilise Spring HATEOAS pour enrichir la réponse d'une API REST avec des liens hypermédias. `EntityModel.of(loan, ...)` crée un modèle d'entité qui encapsule l'objet `loan` et y ajoute des liens. La méthode statique `linkTo(methodOn(LoanController.class).getLoan(id)).withSelfRel()` génère un lien vers la ressource elle-même (le "self" link), en se basant sur la méthode du contrôleur. De même, `linkTo(...).withRel("return")` et `linkTo(...).withRel("renew")` ajoutent des liens pour les actions "return" et "renew", permettant au client de découvrir dynamiquement les opérations possibles sur cette ressource sans avoir de connaissance préalable des URLs. Cela rend l'API plus auto-descriptive et facilite son évolution.

Contrôleur REST complet — Spring Boot

```
@RestController
@RequestMapping("/api/v1/books")
@Validated
public class BookController {

    private final BookService bookService;

    @GetMapping("/{id}")
    public ResponseEntity<BookDto> getBook(@PathVariable UUID id) {
        return bookService.findById(id)
            .map(ResponseEntity::ok)
            .orElseThrow(() -> new BookNotFoundException(id));
    }

    @PostMapping
    public ResponseEntity<BookDto> createBook(
        @Valid @RequestBody CreateBookRequest req) {
        BookDto created = bookService.create(req);
        URI location = URI.create("/api/v1/books/" + created.id());
```

```

        return ResponseEntity.created(location).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<BookDto> updateBook(
        @PathVariable UUID id,
        @Valid @RequestBody UpdateBookRequest req) {
        return ResponseEntity.ok(bookService.update(id, req));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable UUID id) {
        bookService.delete(id);
        return ResponseEntity.noContent().build();
    }
}

```

gRPC — Communication binaire haute performance

Senior

Pourquoi gRPC ?

- **Protobuf** : sérialisation binaire 5-10x plus compacte que JSON.
- **HTTP/2** : multiplexage des connexions, server push, compression des headers.
- **Contrat fort** : le fichier `.proto` est la source de vérité partagée.
- **Streaming** : 4 modes (unary, server-stream, client-stream, bidirectionnel).

Définition du contrat Protobuf (`book.proto`)

```

syntax = "proto3";
package library;
option java_package = "com.example.library.grpc";

service BookService {
    rpc GetBook      (GetBookRequest)  returns (Book);
    rpc CreateBook   (CreateBookRequest) returns (Book);
    // Streaming : Liste paginée de livres
    rpc ListBooks    (ListBooksRequest) returns (stream Book);
    // Streaming bidirectionnel : import en masse
    rpc BulkImport   (stream CreateBookRequest) returns (stream Book);
}

message GetBookRequest { string book_id = 1; }
message ListBooksRequest { int32 page_size = 1; string page_token = 2; }
message CreateBookRequest {
    string title      = 1;

```

```

    string author    = 2;
    string isbn       = 3;
}
message Book {
    string id        = 1;
    string title     = 2;
    string author    = 3;
    string isbn       = 4;
    bool   available = 5;
}

```

Implémentation serveur (streaming server)

```

...

@GrpcService
public class BookGrpcService extends BookServiceGrpc.BookServiceImplBase {

    private final BookRepository bookRepository;

    @Override
    public void getBook(GetBookRequest req,
                       StreamObserver<Book> responseObserver) {
        bookRepository.findById(UUID.fromString(req.getBookId()))
            .map(this::toProto)
            .ifPresentOrElse(
                book -> {
                    responseObserver.onNext(book);
                    responseObserver.onCompleted();
                },
                () -> responseObserver.onError(
                    Status.NOT_FOUND
                        .withDescription("Book not found: " + req.getBookId())
                        .asRuntimeException()
                )
            );
    }

    // Server-side streaming : envoie Les Livres un par un
    @Override
    public void listBooks(ListBooksRequest req,
                          StreamObserver<Book> responseObserver) {
        bookRepository.streamAll().forEach(book -> {
            responseObserver.onNext(toProto(book));
        });
        responseObserver.onCompleted();
    }

    private Book toProto(BookEntity entity) {
        return Book.newBuilder()
            .setId(entity.getId().toString())

```

```

        .setTitle(entity.getTitle())
        .setAuthor(entity.getAuthor())
        .setAvailable(entity.isAvailable())
        .build();
    }
}

```

Messaging asynchrone (Kafka)

Mid

Senior

Publication d'un événement depuis **loan-service**

```

// Événement – record immutable (Java 16+)
public record LoanCreatedEvent(
    UUID    loanId,
    UUID    studentId,
    UUID    bookId,
    Instant createdAt
) {}

@Service
@RequiredArgsConstructor
public class LoanEventPublisher {

    private final KafkaTemplate<String, LoanCreatedEvent> kafka;
    private static final String TOPIC = "loan.created";

    public CompletableFuture<SendResult<String, LoanCreatedEvent>> publishLoanCreated(
        LoanCreatedEvent event) {
        return kafka.send(TOPIC, event.loanId().toString(), event)
            .whenComplete((result, ex) -> {
                if (ex != null) {
                    log.error("Failed to publish LoanCreatedEvent: {}", event.loanId(), ex);
                } else {
                    log.info("Published to partition {} offset {}",
                        result.getRecordMetadata().partition(),
                        result.getRecordMetadata().offset());
                }
            });
    }
}

```

Consommateur dans **notification-service**

```

...

```

```

@Component
@RequiredArgsConstructor
public class LoanCreatedConsumer {

    private final NotificationService notificationService;
    private final MeterRegistry meterRegistry;

    @KafkaListener(
        topics          = "loan.created",
        groupId          = "notification-service",
        containerFactory = "loanEventListenerFactory"
    )
    public void onLoanCreated(
        @Payload LoanCreatedEvent event,
        @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
        @Header(KafkaHeaders.OFFSET) long offset) {

        log.info("Processing LoanCreated [loanId={}, partition={}, offset={}]",
            event.loanId(), partition, offset);

        try {
            notificationService.sendConfirmation(event.studentId(), event.loanId());
            meterRegistry.counter("notifications.sent", "type", "loan-created").increment();
        } catch (Exception e) {
            // Kafka ne commit pas l'offset → retry automatique
            throw new KafkaListenerException("Notification failed", e);
        }
    }
}

```

Configuration Kafka

```

# application.yml
spring:
  kafka:
    bootstrap-servers: kafka:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all          # Attendre acknowledgement de tous les réplicas
      retries: 3
      properties:
        enable.idempotence: true    # Producteur idempotent (pas de doublon)
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      enable-auto-commit: false    # Commit manuel = meilleur contrôle
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer

```

```
value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
properties:
  spring.json.trusted.packages: "com.example.events"
listener:
  ack-mode: MANUAL_IMMEDIATE
```

Mode	Avantage	Limite	Cas d'usage
REST	Simple, universel, CRUD naturel	Couplage temporel, JSON verbeux	API publique, ressources CRUD
gRPC	Binaire (rapide), contrats forts, streaming	Non natif navigateur, outillage spécifique	Appels internes haute fréquence
Kafka	Découplage total, résilience, replay d'events	Broker à opérer, cohérence éventuelle	Notifications, CQRS, audit

Sérialisation et versioning d'API

Junior

Senior

Les composants d'un système distribué communiquent via des **contrats**. Ce contrat doit être stable, documenté et versionné car les services ne sont pas déployés au même rythme.

⚠ Attention

Renommer un champ JSON "innocent" peut casser un autre service déployé à un rythme différent. Toute modification doit être **rétrocompatible ou versionnée explicitement**.

Stratégies de versioning d'API REST

Stratégie	Exemple	Avantage	Limite
URI	<code>/api/v1/books</code>	Très visible, simple à cacher	URL encombrée
Header	<code>Accept: application/vnd.api.v2+json</code>	URL propre, sémantique	Moins découvrable
Query param	<code>?version=2</code>	Simple à tester	Non sémantique REST

✅ Règles d'évolution rétrocompatible

- **Ajouter** des champs optionnels → ✅ rétrocompatible.

- **Renommer / supprimer** un champ → **✗** rupture — période de dépréciation obligatoire.
- **Changer le type** d'un champ → **✗** rupture.
- Pour les événements Kafka : utiliser **Avro + Schema Registry** pour valider les contrats à la publication et à la consommation.

Avro + Schema Registry — Kafka (niveau Senior)

```
// loan-created-event.avsc (Avro Schema)
{
  "type": "record",
  "name": "LoanCreatedEvent",
  "namespace": "com.example.events",
  "fields": [
    { "name": "loanId",    "type": "string" },
    { "name": "studentId", "type": "string" },
    { "name": "bookId",    "type": "string" },
    { "name": "createdAt", "type": "long",   "logicalType": "timestamp-millis" },
    { "name": "channel",   "type": ["null", "string"], "default": null }
    // ↑ Nouveau champ optionnel = rétrocompatible (union avec null + default)
  ]
}
```

```
# application.yml – Avro + Confluent Schema Registry
spring:
  kafka:
    bootstrap-servers: kafka:9092
    producer:
      value-serializer: io.confluent.kafka.serializers.KafkaAvroSerializer
    consumer:
      value-deserializer: io.confluent.kafka.serializers.KafkaAvroDeserializer
    properties:
      schema.registry.url: http://schema-registry:8081
      # BACKWARD = Les nouveaux consommateurs lisent les anciens messages
      # FORWARD  = Les anciens consommateurs lisent les nouveaux messages
      # FULL      = Les deux (recommandé en production)
      avro.compatibility.type: FULL
```

API Gateway

Mid

Senior

L'**API Gateway** est le point d'entrée unique de toutes les requêtes extérieures vers les microservices. Elle centralise les préoccupations transversales.

Mode enfant 🐼

C'est comme le guichet d'accueil d'un grand hôpital : tu ne vas pas directement voir le chirurgien. Tu passes par l'accueil qui vérifie ta carte vitale, sait où t'envoyer, et garde une trace de ta visite.

Responsabilités typiques d'une API Gateway

- 🔄 **Routing** — achemine vers le bon microservice selon le path/host.
- 🛡️ **Authentification / Autorisation** — validation JWT, OAuth2 / OIDC.
- 📊 **Rate limiting** — protection contre les abus (quotas par client).
- 🛠️ **Transformation** — agrégation, adaptation de format.
- 📝 **Logging centralisé** — corrélation de toutes les requêtes entrantes.
- 🔄 **Circuit breaker** au niveau gateway (Resilience4j).

Configuration Spring Cloud Gateway complète

```
spring:
  cloud:
    gateway:
      default-filters:
        - name: RequestRateLimiter
          args:
            redis-rate-limiter.replenishRate: 100    # requêtes/seconde
            redis-rate-limiter.burstCapacity: 200   # pic autorisé
            key-resolver: "#{@userKeyResolver}"

      routes:
        # — loan-service —
        - id: loan-service
          uri: lb://loan-service                    # Load-balanced via Eureka
          predicates:
            - Path=/api/v1/loans/**
          filters:
            - name: CircuitBreaker
              args:
                name: loanServiceCB
                fallbackUri: forward:/fallback/loans
            - AddRequestHeader=X-Gateway-Time, "#{T(java.time.Instant).now()}"

        # — catalog-service —
        - id: catalog-service
          uri: lb://catalog-service
          predicates:
            - Path=/api/v1/books/**
```



```

filters:
  - StripPrefix=0
  - RewritePath=/api/v1/books/(?<segment>.*), /books/${segment}

security:
  oauth2:
    resourceserver:
      jwt:
        jwk-set-uri: http://keycloak:8080/realms/library/protocol/openid-connect/certs

```

Filter de sécurité JWT personnalisé

```

@Component
public class JwtAuthFilter implements GlobalFilter, Ordered {

    private final ReactiveJwtDecoder jwtDecoder;

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String authHeader = exchange.getRequest()
            .getHeaders()
            .getFirst(HttpHeaders.AUTHORIZATION);

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        String token = authHeader.substring(7);
        return jwtDecoder.decode(token)
            .flatMap(jwt -> {
                // Propager les claims vers les services downstream
                ServerHttpRequest mutated = exchange.getRequest().mutate()
                    .header("X-User-Id", jwt.getSubject())
                    .header("X-User-Roles", String.join(",", jwt.getClaimAsStringList("roles")))
                    .build();
                return chain.filter(exchange.mutate().request(mutated).build());
            })
            .onErrorResume(e -> {
                exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
                return exchange.getResponse().setComplete();
            });
    }

    @Override public int getOrder() { return -1; } // avant tous les autres filtres
}

```

⚠️ Anti-pattern : la Gateway intelligente

La gateway doit rester **stupide** (routage, sécurité, observabilité). Évitez d'y placer de la logique métier : vous recréeriez un monolithe distribué difficile à tester et à maintenir.

Service Discovery

Mid

Dans un système distribué, les instances de services démarrent et s'arrêtent dynamiquement. Il est impossible de coder en dur les adresses IP. Le **Service Discovery** résout ce problème en maintenant un registre dynamique.

Client-side Discovery

Le client interroge le registre (Eureka) et choisit lui-même une instance. **Ribbon / Spring Cloud LoadBalancer** fait le load balancing localement.

- ✓ Logique de routage côté client
- ⚠️ Couplage au registre

Server-side Discovery

Le client envoie la requête à un load balancer (AWS ALB, Nginx, Istio) qui interroge le registre et route. Client plus simple.

- ✓ Client léger et découplé
- ⚠️ Composant infra supplémentaire

Configuration Eureka — pom.xml + application.yml

```
...  
  
    <!-- Serveur Eureka -->  
    <dependency>  
        <groupId>org.springframework.cloud</groupId>  
        <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>  
    </dependency>  
  
    <!-- Client (dans chaque microservice) -->  
    <dependency>  
        <groupId>org.springframework.cloud</groupId>  
        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>  
    </dependency>
```

```
# application.yml – dans chaque microservice client
spring:
  application:
    name: loan-service      # identité dans le registre

eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
    fetch-registry: true
    register-with-eureka: true
  instance:
    prefer-ip-address: true
    lease-renewal-interval-in-seconds: 10    # heartbeat
    lease-expiration-duration-in-seconds: 30 # TTL avant désenregistrement
```

🔧 Expert — Alternatives modernes à Eureka

Solution	Contexte	Avantage clé
Kubernetes DNS	Déploiement K8s	Natif, pas de code Java requis
Consul	Multi-cloud, VM + containers	Health checks, KV store, multi-datacenter
Istio Service Mesh	K8s avancé	mTLS, circuit breaker, observabilité sans code

En 2024+, si vous déployez sur Kubernetes, utilisez simplement les **Services K8s** et le DNS cluster interne (`catalog-service.default.svc.cluster.local`). Eureka devient superflu.

Rate Limiting & Throttling

Senior Expert

Le **Rate Limiting** consiste à limiter le nombre de requêtes qu'un client peut effectuer dans une fenêtre de temps donnée. Il protège vos services des abus, des attaques DDoS et de l'utilisation excessive des ressources.

Algorithmes de Rate Limiting

Algorithme	Description	Avantage	Limite
Token Bucket	Seau de jetons rempli à débit constant	Autorise les bursts	Complexe
Fixed Window	Compteur par fenêtre fixe (ex: 1 min)	Simple	Pic en bord de fenêtre

Algorithme	Description	Avantage	Limite
Sliding Window	Fenêtre glissante dans le temps	Plus lisse	Plus de mémoire
Leaky Bucket	Traitement à débit constant	Débit stable	Perd les bursts

Rate Limiting avec Bucket4j (Spring Boot)

```
<dependency>
  <groupId>com.github.vladimir-bukhtoyarov</groupId>
  <artifactId>bucket4j-core</artifactId>
  <version>8.7.0</version>
</dependency>
<dependency>
  <groupId>com.github.vladimir-bukhtoyarov</groupId>
  <artifactId>bucket4j-redis</artifactId>
  <version>8.7.0</version>
</dependency>
```

```
@Component
@RequiredArgsConstructor
public class RateLimitFilter extends OncePerRequestFilter {

    private final BucketProxyManager<String> bucketManager;

    // Limite : 100 req/min par API key
    private final Bandwidth bandwidth = Bandwidth.classic(
        100, Refill.greedy(100, Duration.ofMinutes(1))
    );

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws IOException, ServletException {

        String apiKey = request.getHeader("X-API-Key");
        if (apiKey == null) {
            response.setStatus(HttpStatus.UNAUTHORIZED.value());
            return;
        }

        // Bucket par API key – stocké dans Redis pour la distribution
        Bucket bucket = bucketManager.builder()
            .addLimit(bandwidth)
            .build(apiKey);
```

```

ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);

if (probe.isConsumed()) {
    // Headers standards RFC 6585
    response.addHeader("X-Rate-Limit-Remaining",
        String.valueOf(probe.getRemainingTokens()));
    chain.doFilter(request, response);
} else {
    long waitSeconds = probe.getNanosToWaitForRefill() / 1_000_000_000;
    response.addHeader("X-Rate-Limit-Retry-After-Seconds",
        String.valueOf(waitSeconds));
    response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value()); // 429
}
}
}

```

⚙️ Expert — Rate Limiting distribué avec Redis (Sliding Window)

```

@Component
public class SlidingWindowRateLimiter {

    private final StringRedisTemplate redis;
    private static final int MAX_REQUESTS = 100;
    private static final int WINDOW_SECONDS = 60;

    public boolean isAllowed(String clientId) {
        long now = Instant.now().toEpochMilli();
        long windowStart = now - (WINDOW_SECONDS * 1000L);
        String key = "rate-limit:" + clientId;

        // Script Lua atomique - pas de race condition
        String luaScript = """
            local key = KEYS[1]
            local now = tonumber(ARGV[1])
            local window_start = tonumber(ARGV[2])
            local max_requests = tonumber(ARGV[3])
            local window_ms = tonumber(ARGV[4])

            -- Supprimer les entrées hors de la fenêtre
            redis.call('ZREMRANGEBYSCORE', key, 0, window_start)

            -- Compter les requêtes dans la fenêtre
            local count = redis.call('ZCARD', key)

            if count < max_requests then
                redis.call('ZADD', key, now, now)
            end
        """
    }
}

```

```
        redis.call('EXPIRE', key, window_ms / 1000 + 1)
        return 1
    else
        return 0
    end
end
""";

Long result = redis.execute(
    new DefaultRedisScript<>(luaScript, Long.class),
    List.of(key),
    String.valueOf(now),
    String.valueOf(windowStart),
    String.valueOf(MAX_REQUESTS),
    String.valueOf(WINDOW_SECONDS * 1000)
);

return Long.valueOf(1L).equals(result);
}
```

Le script Lua est exécuté de manière **atomique** par Redis. Pas besoin de verrou distribué car Redis est mono-threadé pour l'exécution des scripts.

✓ Bonnes pratiques

- Toujours retourner **HTTP 429** (Too Many Requests) avec un header `Retry-After`.
- Segmentez les limites : par IP, par utilisateur authentifié, par API key — dans cet ordre croissant de granularité.
- Exposez les headers : `X-Rate-Limit-Limit`, `X-Rate-Limit-Remaining`, `X-Rate-Limit-Reset`.
- Utilisez un stockage distribué (Redis) si vous avez plusieurs instances de gateway.

Distributed Tracing

Mid

Senior

Quand une requête traverse 5 services, comment savoir lequel est lent ? Le **tracing distribué** propage un identifiant unique (**Trace ID**) à travers tous les services pour reconstituer le chemin complet.

Concepts clés

- **Trace ID** : identifiant global de la requête utilisateur de bout en bout.
- **Span** : une étape dans le traitement (ex: appel BDD, appel HTTP distant). Possède un *span parent*.
- **Baggage** : données propagées dans les en-têtes entre services (ex: userId, tenantId).
- **Sampling** : on ne trace pas 100% des requêtes en prod (coût) — on échantillonne (5% à 10%).

Configuration Micrometer Tracing + OpenTelemetry + Zipkin

```

<!-- pom.xml -->
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-zipkin</artifactId>
</dependency>

```

```

# application.yml
management:
  tracing:
    sampling:
      probability: 1.0  # 100% en dev, ~5% en prod
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans

logging:
  pattern:
    level: "%5p [%${spring.application.name},%X{traceId},%X{spanId}]"

```

Span personnalisé avec annotation

```

@Service
public class LoanApplicationService {

  private final Tracer tracer;

  // @Observed crée automatiquement un span
  @Observed(name = "loan.create",
    contextualName = "creating-loan",
    lowCardinalityKeyValues = {"service", "loan"})
  public LoanDto createLoan(CreateLoanRequest request) {
    // ... Logique métier
  }

  // Span manuel pour plus de contrôle
  public void processReturns(List<UUID> loanIds) {
    Span span = tracer.nextSpan()
      .name("process-returns")

```

```

        .tag("count", String.valueOf(loanIds.size()))
        .start();

    try (Tracer.SpanInScope ignored = tracer.withSpan(span)) {
        loanIds.forEach(this::processReturn);
    } catch (Exception e) {
        span.tag("error", e.getMessage());
        throw e;
    } finally {
        span.end();
    }
}
}

```

✓ Bonne pratique

Toujours inclure le `traceId` dans les logs et dans les réponses d'erreur (header `X-Trace-Id`). C'est la clé pour déboguer un incident en production en **5 minutes** au lieu de 5 heures.

Event Sourcing (introduction)

Senior

Expert

Au lieu de stocker *l'état actuel* d'une entité, on stocke la **séquence de tous les événements** qui ont conduit à cet état. L'état courant est reconstruit en rejouant les événements.

Mode enfant 🐼

Plutôt que de noter "solde = 150€", tu notes toutes les opérations : "dépôt 200€", "retrait 50€". Tu peux retrouver le solde à n'importe quel moment dans le passé.

Modélisation en Java — Sealed interfaces + Records

```

// Les événements sont immuables et typés précisément
public sealed interface LoanEvent permits
    LoanRequestedEvent, LoanApprovedEvent, LoanReturnedEvent,
    LoanRenewedEvent, LoanOverdueEvent {}

public record LoanRequestedEvent(
    UUID loanId, UUID studentId, UUID bookId, Instant at) implements LoanEvent {}

public record LoanApprovedEvent(
    UUID loanId, Instant dueDate, Instant at) implements LoanEvent {}

```



```

public record LoanReturnedEvent(
    UUID loanId, Instant returnedAt) implements LoanEvent {}

// Agrégat reconstruit depuis Les événements
public class Loan {
    private UUID id;
    private LoanStatus status;
    private Instant dueDate;
    private List<LoanEvent> uncommittedEvents = new ArrayList<>();

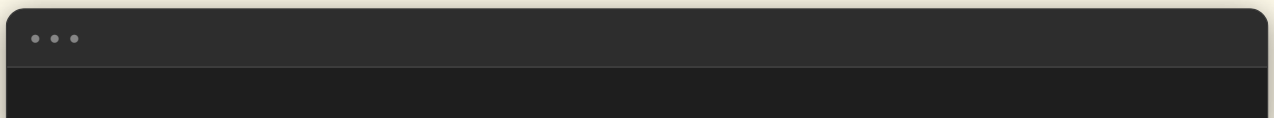
    // Factory – reconstruction depuis L'event store
    public static Loan reconstitute(List<LoanEvent> events) {
        Loan loan = new Loan();
        events.forEach(loan::apply);
        return loan;
    }

    // Commande – génère des événements
    public void approve(Instant dueDate) {
        if (status != LoanStatus.REQUESTED) {
            throw new IllegalStateException("Loan is not in REQUESTED state");
        }
        LoanApprovedEvent event = new LoanApprovedEvent(id, dueDate, Instant.now());
        apply(event);
        uncommittedEvents.add(event); // à persister dans L'event store
    }

    // Apply – état interne muté par L'événement
    private void apply(LoanEvent event) {
        switch (event) {
            case LoanRequestedEvent r -> {
                this.id = r.loanId();
                this.status = LoanStatus.REQUESTED;
            }
            case LoanApprovedEvent a -> {
                this.status = LoanStatus.APPROVED;
                this.dueDate = a.dueDate();
            }
            case LoanReturnedEvent ignored -> this.status = LoanStatus.RETURNED;
            case LoanRenewedEvent r -> this.dueDate = r.newDueDate();
            case LoanOverdueEvent ignored -> this.status = LoanStatus.OVERDUE;
        }
    }
}

```

🔧 Expert — Snapshots pour éviter la reconstruction coûteuse



```
// Sans snapshot : rejouer 10 000 événements à chaque lecture = lent
// Avec snapshot : rejouer depuis le dernier snapshot seulement

@Component
public class LoanRepository {

    private final EventStore eventStore;
    private final SnapshotStore snapshotStore;
    private static final int SNAPSHOT_THRESHOLD = 50; // snapshot tous les 50 events

    public Loan findById(UUID loanId) {
        // 1. Charger le dernier snapshot (s'il existe)
        Optional<LoanSnapshot> snapshot = snapshotStore.findLatest(loanId);

        long fromVersion = snapshot.map(LoanSnapshot::version).orElse(0L);
        Loan loan = snapshot.map(LoanSnapshot::toLoan).orElseGet(Loan::new);

        // 2. Rejouer uniquement les événements après le snapshot
        List<LoanEvent> events = eventStore.findEventsAfter(loanId, fromVersion);
        events.forEach(loan::apply);

        return loan;
    }

    public void save(Loan loan) {
        long version = eventStore.appendEvents(loan.getId(), loan.getUncommittedEvents());

        // 3. Créer un snapshot si le seuil est atteint
        if (version % SNAPSHOT_THRESHOLD == 0) {
            snapshotStore.save(new LoanSnapshot(loan.getId(), version, loan.toState()));
        }

        loan.clearUncommittedEvents();
    }
}
```

⚠ Quand utiliser Event Sourcing ?

L'Event Sourcing est puissant mais complexe. À réserver aux **domaines où l'audit et l'historique complet sont critiques** (finance, compliance, logistique). Pour un CRUD simple, c'est un over-engineering qui complique tout.

Consumer-Driven Contract Testing

Senior

Expert

Dans un système distribué, comment s'assurer qu'un changement côté *provider* ne casse pas un *consumer* ? Le **Contract Testing** (Pact / Spring Cloud Contract) formalise les attentes des consommateurs en contrats vérifiables automatiquement.

Principe

1. Le **consumer** écrit les interactions attendues (requête → réponse).
2. Ces interactions génèrent un **contrat Pact** (JSON).
3. Le **provider** joue le contrat contre sa vraie implémentation — en CI.
4. Si le provider casse le contrat, le build échoue **avant le déploiement**.

Côté Consumer — Test Pact (loan-service → catalog-service)

```
@ExtendWith(PactConsumerTestExt.class)
@PactTestFor(providerName = "catalog-service", port = "8081")
class CatalogClientContractTest {

    @Pact(consumer = "loan-service")
    public RequestResponsePact availabilityPact(PactDslWithProvider builder) {
        return builder
            .given("book d290f1ee exists and is available")
            .uponReceiving("GET availability for book d290f1ee")
                .path("/internal/books/d290f1ee-6c54-4b01-90e6-d701748f0851/availability")
                .method("GET")
            .willRespondWith()
                .status(200)
                .headers(Map.of("Content-Type", "application/json"))
                .body(new PactDslJsonBody()
                    .booleanValue("available", true)
                    .integerType("copies", 3))
            .toPact();
    }

    @Test
    @PactTestFor(pactMethod = "availabilityPact")
    void testGetAvailability(MockServer mockServer) {
        CatalogClient client = new CatalogClient(mockServer.getUrl());
        AvailabilityResponse response = client.getAvailability(
            UUID.fromString("d290f1ee-6c54-4b01-90e6-d701748f0851")
        );

        assertThat(response.available()).isTrue();
        assertThat(response.copies()).isEqualTo(3);
    }
}
```

Côté Provider — Vérification du contrat

```
@Provider("catalog-service")
@PactBroker(url = "http://pact-broker:9292")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class CatalogServiceContractVerificationTest {

    @LocalServerPort
    private int port;

    @BeforeEach
    void setup(PactVerificationContext context) {
        context.setTarget(new HttpTestTarget("localhost", port));
    }

    @TestTemplate
    @ExtendWith(PactVerificationInvocationContextProvider.class)
    void verifyPact(PactVerificationContext context) {
        context.verifyInteraction(); // rejoue Le contrat du consumer
    }

    // Setup des données de test pour L'état "book d290f1ee exists and is available"
    @State("book d290f1ee exists and is available")
    void bookExists() {
        bookRepository.save(new Book(
            UUID.fromString("d290f1ee-6c54-4b01-90e6-d701748f0851"),
            "Clean Architecture", "Robert C. Martin", true, 3
        ));
    }
}
```

✓ Workflow recommandé en CI/CD

1. Consumer publie le contrat Pact dans le **Pact Broker**.
2. Le pipeline du Provider tire le contrat et joue la vérification.
3. Le Provider peut être déployé uniquement si **can-i-deploy** passe (`pact-broker can-i-deploy`).

Résultat : vous pouvez déployer chaque service indépendamment avec la **confiance** que les contrats sont respectés.

Quiz de fin — Jour 3

1. Pourquoi un retry peut-il aggraver un incident ?

► [Voir la réponse](#)

2. Dans quel cas le messaging asynchrone est-il préférable à REST ?

► [Voir la réponse](#)

3. Pourquoi le versioning est-il central dans un système distribué ?

► [Voir la réponse](#)

4. Quelle est la différence entre un Trace ID et un Span ID ?

► [Voir la réponse](#)

5. Quelle "fallacy" du distribué explique qu'on oublie souvent de gérer les timeouts ?

► [Voir la réponse](#)

6. Quelle est la différence entre PACELC et CAP ? Donnez un exemple d'application de chaque.

► [Voir la réponse](#)

7. Qu'est-ce que le Consumer-Driven Contract Testing et quel problème résout-il ?

► [Voir la réponse](#)

8. (Expert) Pourquoi le script Lua est-il utilisé pour le Rate Limiting avec Redis ?

► [Voir la réponse](#)

← Jour 2

Jour 4 →